API SECURITY RISK ASSESSMENT REPORT

Project Title: API Security Risk Assessment (FUTURE_CS_03)

Author: Furrell Jordan Meas

Date: 18 April 2026

Email: furrelljordanmeas@gmail.com

Phone: 060 695 9079

TABLE OF CONTENTS

1. Executive Summary 2
2. Methodology 2
2.1 Tools Used 2
2.2 Testing Approach 2
2.3 Frameworks Referenced 3
3. API Inventory 3
4. Security Findings 3
4.1 Broken Object Level Authorisation 4
4.2 Broken Function Level Authorization 6
4.3 Broken Object Property Level Authorization 7
4.4 Improper Inventory Management 8
5. Risk Rating Summary 9
6. Recommendations & Remediation 10
7. Conclusion 10

1. Executive Summary

This assessment evaluates the security posture of the publicly accessible REST API provided by JSONPlaceholder. Testing was conducted using manual techniques and tools such as Insomnia and browser developer tools.

The primary objective was to identify common API vulnerabilities in alignment with the OWASP API Security Top 10 (2023) framework.

Summary of Findings

A total of 4 key vulnerabilities were identified:

- **High Severity (2):**
  - Broken Object Level Authorization (BOLA)

    - Broken Function Level Authorization
- **Medium Severity (1):**
    - Broken Object Property Level Authorization
- **Low Severity (1):**
    - Improper Inventory Management

Key Security Risks

The most critical risks stem from the absence of authentication and authorization controls, resulting in:

- Unrestricted access to API resources
- Unauthorized execution of CRUD operations
- Excessive data exposure in API responses

Business Impact

If deployed in a production environment, these vulnerabilities could result in:

- Exposure of sensitive user data (PII)
- Loss of data integrity through unauthorized modifications
- Increased susceptibility to automated attacks and exploitation

2. Methodology

This assessment followed a structured manual testing approach to evaluate API security controls.

2.1 Tools Used

- **Insomnia** – Primary API testing tool used to send HTTP requests and analyze responses

- **Browser Developer Tools (Inspector tab)** – Used to inspect network traffic and API responses
- **Web browser** – Used to access endpoints and verify public API behavior

2.2 Testing Approach

1. Endpoint Discovery

- Identified API endpoints through documentation and manual enumeration
- Confirmed access to endpoints such as /users, /posts, /comments, /todos, /albums, /photos

2. Authentication Testing

- Tested for authentication requirements (API keys, tokens)
- Determined that all endpoints are publicly accessible without credentials

3. Authorization Testing

- Manipulated object identifiers (IDs) to test access controls
- Verified lack of user-level access restrictions

4. Functionality Testing

- Executed CRUD operations (GET, POST, PUT, DELETE)
- Observed absence of role-based restrictions

5. Data Exposure Analysis

- Reviewed API responses for sensitive or excessive data exposure
- Assessed adherence to data minimization principles

6. Security Control Testing

- Evaluated rate limiting, input validation, and error handling
- Observed absence of throttling and request controls

2.3 Frameworks Referenced

- OWASP API Security Top 10 (2023)

3. API Inventory

Resource	Endpoint	Supported Methods	Auth Required	Description
Posts	`/posts`	GET, POST, PUT, PATCH, DELETE	None	Represents blog posts with userId, title, and body fields
Comments	`/comments`	GET, POST, PUT, PATCH, DELETE	None	Comments linked to posts, including name, email, and body
Albums	`/albums`	GET, POST, PUT, PATCH, DELETE	None	User photo albums associated with a userId
Photos	`/photos`	GET, POST, PUT, PATCH, DELETE	None	Image metadata including URLs, thumbnails, and albumId
Todos	`/todos`	GET, POST, PUT, PATCH, DELETE	None	Task management items with completion status
Users	`/users`	GET, POST, PUT, PATCH, DELETE	None	User profile data including address, company, and contact information

4. Security Findings

4.1 Broken Object Level Authorisation

Severity: High
OWASP Category: API1: Broken Object Level Authorization

Description

The API fails to enforce object-level access control, allowing users to access resources belonging to other users by manipulating object identifiers.

Example:

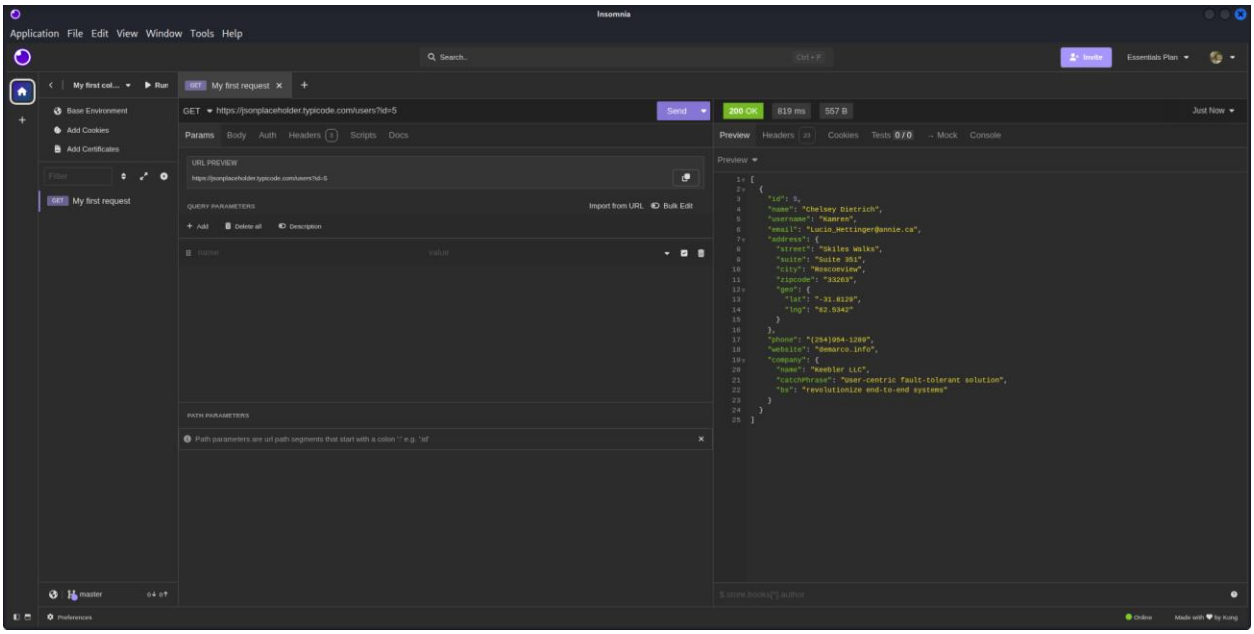
```
GET /users/1
GET /users/2
```

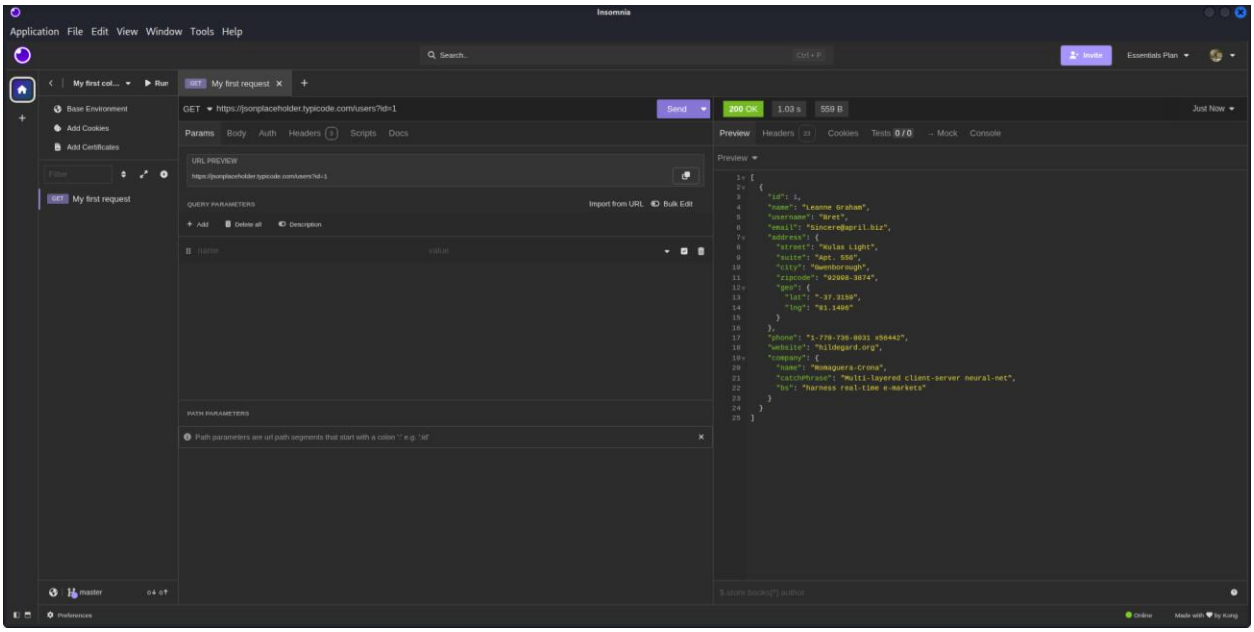
Impact

- Unauthorized access to sensitive data

- Privacy violations (PII exposure)
- Large-scale data scraping

Proof of Concept





Recommendation

- Enforce object-level authorization checks
- Validate resource ownership per request
- Implement JWT or OAuth2 authentication
- Replace sequential IDs with UUIDs
- Use centralized authorization middleware

4.2 Broken Function Level Authorization

Severity: High
OWASP Category: API5: Broken Function Level Authorization

Description

The API does not restrict access to sensitive operations, allowing unauthorized users to perform privileged actions.

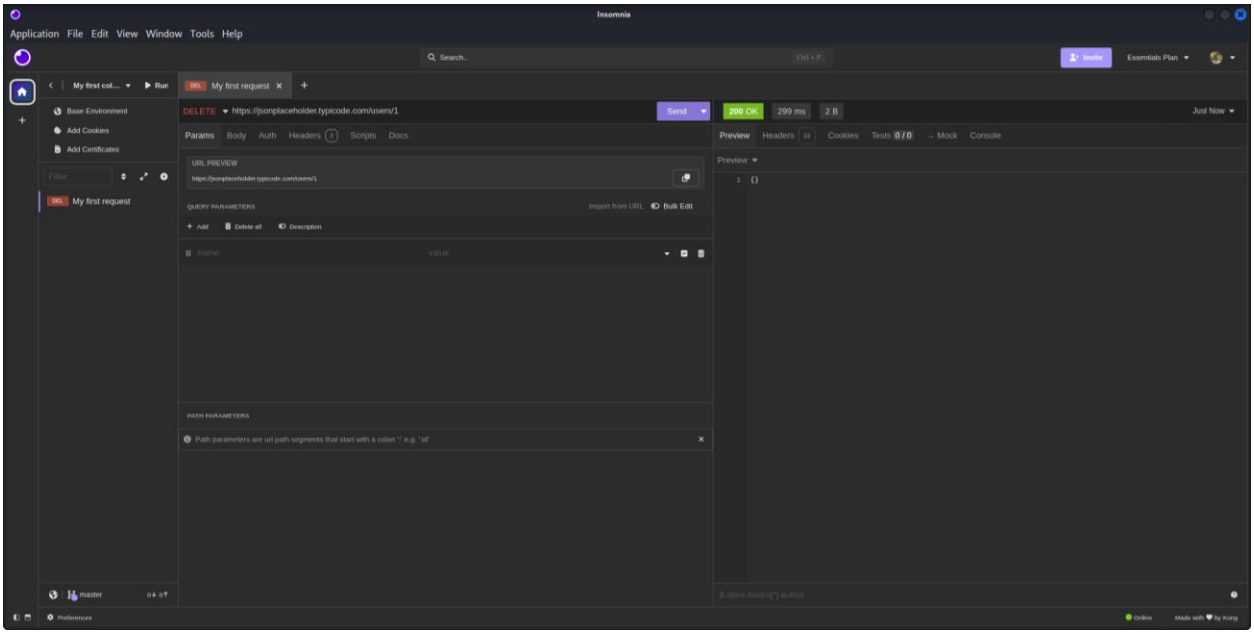
Example:

```
POST /posts
PUT /posts/1
DELETE /posts/1
```

Impact

- Unauthorized data modification or deletion
- Privilege escalation
- Compromise of system integrity

Proof of Concept



Recommendation

- Implement Role-Based Access Control (RBAC)
- Enforce authorization at service/controller level
- Separate user and admin functionalities
- Apply a deny-by-default access model

4.3 Broken Object Property Level Authorization

Severity: Medium
OWASP Category: API3: Broken Object Property Level Authorization

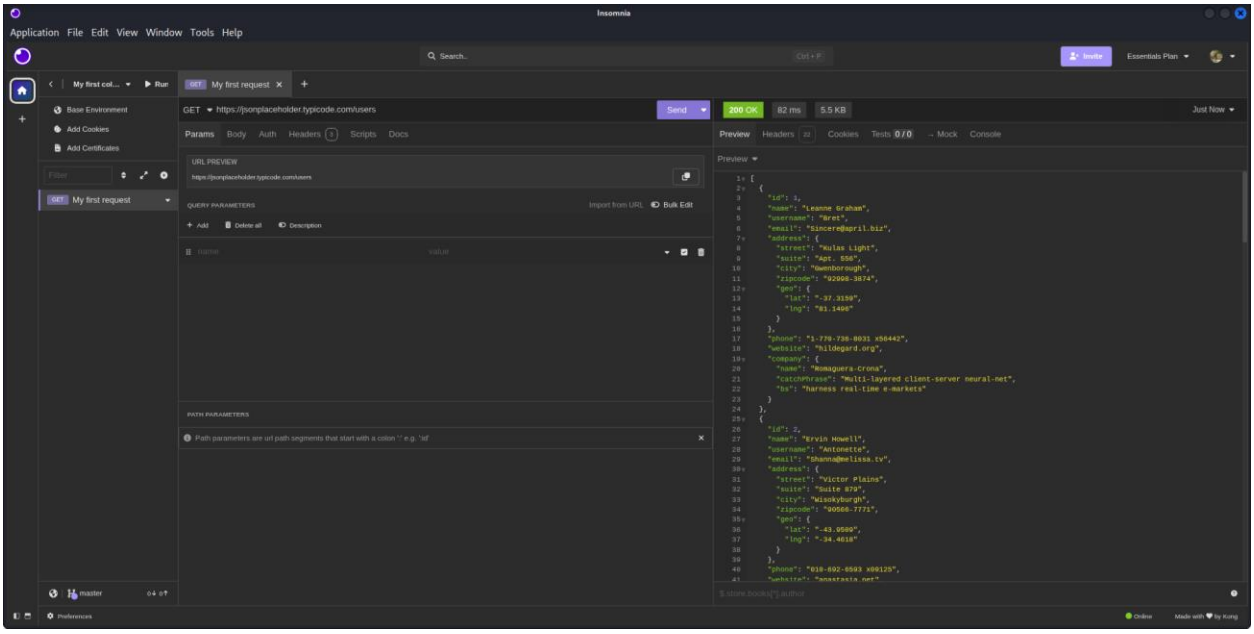
Description

The API exposes sensitive object properties without proper filtering or restriction.

Impact

- Exposure of sensitive user information
- Increased risk of phishing/social engineering
- Violations of data minimization principles

Proof of Concept



Recommendation

- Implement field-level access control
- Use DTOs or view models
- Return only necessary data fields
- Apply serialization filtering (whitelisting)

4.4 Improper Inventory Management

Severity: Low
OWASP Category: API9: Improper Inventory Management

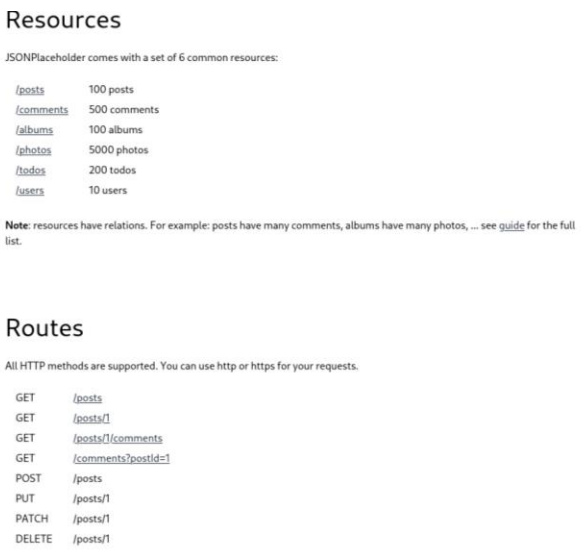
Description

The API lacks proper inventory control, exposing multiple endpoints without clear governance.

Impact

- Expanded attack surface
- Exposure of deprecated or undocumented endpoints
- Easier reconnaissance for attackers

Proof of Concept



Resources

JSONPlaceholder comes with a set of 6 common resources:

/posts	100 posts
/comments	500 comments
/albums	100 albums
/photos	5000 photos
/todos	200 todos
/users	10 users

Note: resources have relations. For example: posts have many comments, albums have many photos, ... see guide for the full list.

Routes

All HTTP methods are supported. You can use http or https for your requests.

GET	/posts
GET	/posts/1
GET	/posts/1/comments
GET	/comments?postId=1
POST	/posts
PUT	/posts/1
PATCH	/posts/1
DELETE	/posts/1

Recommendation

- Maintain a centralized API inventory
- Remove deprecated endpoints
- Enforce API versioning (e.g., `/api/v1/`)

- Secure API documentation (e.g., Swagger)
- Monitor and log endpoint usage

5. Risk Rating Summary

Vulnerability	Severity	OWASP Category
Broken Object Level Authorization	High	API1
Broken Function Level Authorization	High	API5
Broken Object Property Level Authorization	Medium	API3
Improper Inventory Management	Low	API9

6. Recommendations & Remediation

Authentication & Authorization

- Implement OAuth2 or JWT-based authentication
- Enforce strict object-level authorization
- Apply RBAC or ABAC models

Data Protection

- Apply data minimization principles
- Use DTOs to control response structure
- Restrict sensitive fields by default

API Hardening

- Deploy an API Gateway
- Implement rate limiting and throttling
- Disable unnecessary HTTP methods

Governance & Inventory

- Maintain a formal API registry

- Remove unused endpoints
- Secure API documentation

7. Conclusion

The API demonstrates **moderate to high security risk**, primarily due to missing authentication and authorization controls.

High-severity vulnerabilities—particularly BOLA and Broken Function Level Authorization—require immediate remediation to prevent unauthorized data access and system compromise.